Rheinland-Pfälzische
Technische Universität
Kaiserslautern
Landau

Optimizing Error Messages: The Impact of Optimized Error Messages in Teaching F#

Bachelor's Thesis

by

Jakob Meyerhöfer

February 13, 2025

University of Kaiserslautern-Landau
Department of Computer Science
67663 Kaiserslautern
Germany

Examiner: Prof. Dr. Ralf Hinze
Sebastian Schloßer, M.Sc. (Sebastian Schweizer)

Declaration of Authorship

I hereby declare that the work I have submitted with the title „Optimizing Error Messages: The Impact of Optimized Error Messages in Teaching F#“ was independently written by me. I have fully disclosed all sources and aids used and have identified as quotations all passages of the work — including tables and figures — taken verbatim or in substance from other works or the internet, indicating the respective sources.

Kaiserslautern, on 13.2.2025

Jakob Meyerhöfer

Abstract

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Summary

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Contents

1. Introduction **1**
- 1.1. Motivation . . . 1
- 1.2. Research Problem and Objectives . . . 1
- 1.3. Research Questions . . . 1
- 1.4. Research Structure . . . 1

2. Related Work **3**
- 2.1. Challenges for Novice Programmers . . . 3
- 2.2. Role of Error Messages in Learning . . . 3
- 2.3. Existing Approaches to Improving Error Messages . . . 3
- 2.4. Educational Feedback Theories . . . 3

3. Analysis of Previous Semester Data **5**
- 3.1. Data Collection and Methodology . . . 5
- 3.2. Compilation Success Rates . . . 5
- 3.3. Test Pass Rates . . . 5
- 3.4. Insights and Patterns . . . 5

4. Problem Analysis: Issues with Existing Error Messages **7**
- 4.1. Examples of Current Error Messages . . . 7
  - 4.1.1. Example 1 - `digitSum` . . . 8
  - 4.1.2. Example 2 - `sortedDigits` . . . 12
- 4.2. Information Gaps in Error Outputs . . . 15
- 4.3. Challenges for Novices in Deriving Test Information . . . 15

5. Assertify: A Class for Improving Error Messages **17**
- 5.1. Objectives and Design of Assertify . . . 17
- 5.2. Implementation Details and Wrappers . . . 17
- 5.3. Error Message Improvements (Before and After Comparisons) . . . 17
- 5.4. Usability and Design Considerations . . . 18

6. TODO: Further Methodology (unclear what comes next) **19**

7. Evaluation and Results **21**
- 7.1. Evaluation Methodology . . . 21
- 7.2. Metrics for Success . . . 21
- 7.3. Quantitative Analysis . . . 21
- 7.4. Qualitative Insights . . . 21
- 7.5. Comparison to Previous Results . . . 22

8. Conclusion **23**
8.1. Summary of Contributions 23
8.2. Key Findings 23
8.3. Suggestions for Future Work 23

9. Discussion **25**
9.1. Interpretation of Results 25
9.2. Impact on Learning Process 25
9.3. Strengths and Limitations of Approach 25
9.4. Recommendations for Future Development 25

List of Figures **27**

A. My Code **29**

B. My Ideas **31**

Bibliography **33**

1. Introduction

1.1. Motivation

TODO

Explain why improving error messages is important for novice programmers.

1.2. Research Problem and Objectives

TODO

Define the problem of unclear error messages and the goals of the thesis.

1.3. Research Questions

TODO

List the key questions this thesis seeks to answer.

1.4. Research Structure

TODO

Briefly outline the chapters and their purpose.

2. Related Work

TODO

Apparently there is something to do.

[Tol12]

2.1. Challenges for Novice Programmers

TODO

Describe common difficulties novices face when learning programming.

2.2. Role of Error Messages in Learning

TODO

Discuss how error messages impact the debugging and learning process.

2.3. Existing Approaches to Improving Error Messages

TODO

Review current solutions and their limitations.

2.4. Educational Feedback Theories

TODO

Summarize relevant feedback theories from educational research.

3. Analysis of Previous Semester Data

TODO

Apparently there is something to do.

3.1. Data Collection and Methodology

TODO

Describe how you gathered and analyzed semester data.

3.2. Compilation Success Rates

TODO

Present the success rates for project compilation.

3.3. Test Pass Rates

TODO

Analyze the percentage of passed tests.

3.4. Insights and Patterns

TODO

Highlight key patterns and findings from the data.

4. Problem Analysis: Issues with Existing Error Messages

TODO

Apparently there is something to do.

4.1. Examples of Current Error Messages

TODO

Show examples of unhelpful or ambiguous error messages.

4.1.1. Example 1 - `digitSum`

Implement a method `digitSum: Nat -> Nat`, that takes a natural number `n` as input and returns its digit sum. Examples:

```
digitSum 0N = 0N
digitSum 123N = 6N
digitSum 1234N = 10N
digitSum 4711N = 13N
```

```
let rec digitSum (n: Nat): Nat =
    if n = 0N then
        1N
    else
        (n % 10N) + digitSum (n / 10N)
```

Listing 4.1: Student submission for `digitSum`

```
let rec digitSumSolution (n: Nat): Nat =
    if n = 0N then
        0N
    else
        (n % 10N) + digitSumSolution (n / 10N)
```

Listing 4.2: Sample solution for `digitSum`

Example Tests - digitSum

```
    [<TestMethod>] [<Timeout(1000)>]
    member _.``a) digitSum Examples`` (): unit =
        test <@ digitSum 123N  = 6N @>
        test <@ digitSum 1234N = 10N @>
        test <@ digitSum 42N   = 6N @>
        test <@ digitSum 105N  = 6N @>
        test <@ digitSum 0N    = 0N @>
        test <@ digitSum 4711N = 13N @>
```

Listing 4.3: Example tests for digitSum

```
a) digitSum Examples (388ms): Error Message: Test method Tests+Tests.a)
↪  digitSum Examples threw exception:
      Swensen.Unquote.AssertionFailedException: Test failed:

      StudentSubmission.digitSum 123N = 6N
      StudentSubmission.digitSum 123N = 6N
      7N = 6N
      false

      Stack Trace:
          at Tests.Tests.a) digitSum Examples() in path\to\Tests.fs:line 57
         at System.RuntimeMethodHandle.InvokeMethod(Object target, Void**
         ↪  arguments, Signature sig, Boolean isConstructor)
         at System.Reflection.MethodBaseInvoker.InvokeWithNoArgs(Object obj,
         ↪  BindingFlags invokeAttr)
```

Listing 4.4: Output of examples tests for digitSum

Key Information Example Tests - digitSum

Tested expression	digitSum 123N
Expected result	6N
Actual result	7N

Random Tests - `digitSum`

```
    [<TestMethod>] [<Timeout(5000)>]
    member _.``a) digitSum Random`` (): unit =
        Check.One (
            { Config.QuickThrowOnFailure with EndSize = 1000 },
            fun (n: Nat) -> Assert.AreEqual<Nat> (
                digitSumSolution n, digitSum n
            )
        )
```

Listing 4.5: Random tests for `digitSum`

```
a) digitSum Random (74ms): Error Message: Test method Tests+Tests.a) digitSum
↪  Random threw exception:
      System.Exception: Falsifiable, after 1 test (1 shrink) (StdGen (
      ↪  1765833463, 297438798)):
      Original:
      11N
      Shrunk:
      0N
      with exception:
      Microsoft.VisualStudio.TestTools.UnitTesting.AssertFailedException:
      ↪  Assert.AreEqual failed.
      Expected:<0>. Actual:<1>.
         at Microsoft.VisualStudio.TestTools.UnitTesting.Assert.
         ↪  ThrowAssertFailed(String assertionName, String message) in /_/src/
         ↪  TestFramework/TestFramework/Assertions/Assert.cs:line 60
         at Microsoft.VisualStudio.TestTools.UnitTesting.Assert.AreEqual[T](T
         ↪  expected, T actual, IEqualityComparer`1 comparer, String message,
         ↪  Object[] parameters) in /_/src/TestFramework/TestFramework/Assert
      ions/Assert.AreEqual.cs:line 201
         at Microsoft.VisualStudio.TestTools.UnitTesting.Assert.AreEqual[T](T
         ↪  expected, T actual) in /_/src/TestFramework/TestFramework/
         ↪  Assertions/Assert.AreEqual.cs:line 34
         at Tests.a) digitSum Random@69.Invoke(Nat n) in path\to\Tests.fs:line
         ↪  69
         at FsCheck.Testable.evaluate[a,b](FSharpFunc`2 body, a a)

      Stack Trace:
          at Microsoft.FSharp.Core.PrintfModule.PrintFormatToStringThenFail@
          ↪  1448.Invoke(String message)
         at <StartupCode$FsCheck>.$Runner.get_throwingRunner@382.FsCheck.
         ↪  IRunner.OnFinished(String name, TestResult testResult)
         at FsCheck.Runner.runner[a](Config config, a prop)
         at FsCheck.Runner.check[a](Config config, a p)
         at FsCheck.Check.One[Testable](Config config, Testable property)
         at Tests.Tests.a) digitSum Random() in path\to\Tests.fs:line 68
         at System.RuntimeMethodHandle.InvokeMethod(Object target, Void**
         ↪  arguments, Signature sig, Boolean isConstructor)
         at System.Reflection.MethodBaseInvoker.InvokeWithNoArgs(Object obj,
         ↪  BindingFlags invokeAttr)
```

Listing 4.6: Output of random tests for `digitSum`

Key Information Random Tests - `digitSum`

Tested expression	`a) digitSum Random` `Original: 11N` } `digitSum 11N`
Expected result	`Expected:<0>` $\Longrightarrow$ `0` (actually `0N`)
Actual result	`Actual:<1>` $\Longrightarrow$ `1` (actually `1N`)

4.1.2. Example 2 - `sortedDigits`

Implement a method `sortedDigits: Nat -> bool`, that takes a natural number `n` as input and returns `true` if the digits are in ascending order and `false` otherwise. Examples:

```
sortedDigits 0N = true          sortedDigits 1111N = true
sortedDigits 5N = true          sortedDigits 1101N = false
sortedDigits 159N = true        sortedDigits 543N = false
```

```
let rec sortedDigits (n: Nat): bool =
    if n = 0N then
        true
    else
        let frontSorted: bool = sortedDigits (n / 10N)
        let lastDigit: Nat = n % 10N
        let secondLastDigit: Nat = (n / 10N) % 10N
        frontSorted && secondLastDigit < lastDigit
```

Listing 4.7: Student submission for `sortedDigits`

```
let rec sortedDigitsSolution (n: Nat): bool =
    if n = 0N then
        true
    else
        let frontSorted: bool = sortedDigitsSolution (n / 10N)
        let lastDigit: Nat = n % 10N
        let secondLastDigit: Nat = (n / 10N) % 10N
        frontSorted && secondLastDigit <= lastDigit
```

Listing 4.8: Sample solution for `sortedDigits`

Example Tests - `sortedDigits`

```
    [<TestMethod>] [<Timeout(1000)>]
    member _.``b) sortedDigits Examples`` (): unit =
        test <@ sortedDigits 0N    = true @>
        test <@ sortedDigits 5N    = true @>
        test <@ sortedDigits 159N  = true @>
        test <@ sortedDigits 1111N = true @>
        test <@ sortedDigits 42N   = false @>
        test <@ sortedDigits 543N  = false @>
        test <@ sortedDigits 1101N = false @>
```

Listing 4.9: Example tests for `sortedDigits`

```
b) sortedDigits Examples (4ms): Error Message: Test method Tests+Tests.b)
↪  sortedDigits Examples threw exception:
      Swensen.Unquote.AssertionFailedException: Test failed:

      StudentSubmission.sortedDigits 1111N = true
      StudentSubmission.sortedDigits 1111N = true
      false = true
      false

      Stack Trace:
          at Tests.Tests.b) sortedDigits Examples() in path\to\Tests.fs:line 80
         at System.RuntimeMethodHandle.InvokeMethod(Object target, Void**
         ↪  arguments, Signature sig, Boolean isConstructor)
         at System.Reflection.MethodBaseInvoker.InvokeWithNoArgs(Object obj,
         ↪  BindingFlags invokeAttr)
```

Listing 4.10: Output of examples tests for `sortedDigits`

Key Information Example Tests - `sortedDigits`

Tested expression	`sortedDigits 123N`
Expected result	`true`
Actual result	`false`

Random Tests - `sortedDigits`

```
    [<TestMethod>] [<Timeout(5000)>]
    member _.``b) sortedDigits Random`` (): unit =
        Check.One (
            { Config.QuickThrowOnFailure with EndSize = 1000 },
            fun (n: Nat) -> Assert.AreEqual<bool> (
                sortedDigitsSolution n, sortedDigits n
            )
        )
```

Listing 4.11: Random tests for `sortedDigits`

```
b) sortedDigits Random (3ms): Error Message: Test method Tests+Tests.b)
↪  sortedDigits Random threw exception:
      System.Exception: Falsifiable, after 5 tests (0 shrinks) (StdGen (
      ↪  867569077, 297439050)):
      Original:
      33N
      with exception:
      Microsoft.VisualStudio.TestTools.UnitTesting.AssertFailedException:
      ↪  Assert.AreEqual failed.
      Expected:<True>. Actual:<False>.
         at Microsoft.VisualStudio.TestTools.UnitTesting.Assert.
         ↪  ThrowAssertFailed(String assertionName, String message) in /_/src/
         ↪  TestFramework/TestFramework/Assertions/Assert.cs:line 60
         at Microsoft.VisualStudio.TestTools.UnitTesting.Assert.AreEqual[T](T
         ↪  expected, T actual, IEqualityComparer`1 comparer, String message,
         ↪  Object[] parameters) in /_/src/TestFramework/TestFramework/Assert
      ions/Assert.AreEqual.cs:line 201
         at Microsoft.VisualStudio.TestTools.UnitTesting.Assert.AreEqual[T](T
         ↪  expected, T actual) in /_/src/TestFramework/TestFramework/
         ↪  Assertions/Assert.AreEqual.cs:line 34
         at Tests.b) sortedDigits Random@90.Invoke(Nat n) in path\to\Tests.fs:
         ↪  line 90
         at FsCheck.Testable.evaluate[a,b](FSharpFunc`2 body, a a)

      Stack Trace:
          at Microsoft.FSharp.Core.PrintfModule.PrintFormatToStringThenFail@
          ↪  1448.Invoke(String message)
         at <StartupCode$FsCheck>.$Runner.get_throwingRunner@382.FsCheck.
         ↪  IRunner.OnFinished(String name, TestResult testResult)
         at FsCheck.Runner.runner[a](Config config, a prop)
         at FsCheck.Runner.check[a](Config config, a p)
         at FsCheck.Check.One[Testable](Config config, Testable property)
         at Tests.Tests.b) sortedDigits Random() in path\to\Tests.fs:line 89
         at System.RuntimeMethodHandle.InvokeMethod(Object target, Void**
         ↪  arguments, Signature sig, Boolean isConstructor)
         at System.Reflection.MethodBaseInvoker.InvokeWithNoArgs(Object obj,
         ↪  BindingFlags invokeAttr)
```

Listing 4.12: Output of random tests for `sortedDigits`

Key Information Random Tests - `sortedDigits`

Tested expression	`b) sortedDigits Random` / `Original: 33N` } `sortedDigits 33N`
Expected result	`Expected:<True>` $\Longrightarrow$ `True` (actually `true`)
Actual result	`Actual:<False>` $\Longrightarrow$ `False` (actually `false`)

4.2. Information Gaps in Error Outputs

TODO

Analyze what critical information is missing in these messages.

4.3. Challenges for Novices in Deriving Test Information

TODO

Explain why deciphering test expectations is hard for beginners.

5. Assertify: A Class for Improving Error Messages

TODO

Apparently there is something to do.

5.1. Objectives and Design of Assertify

TODO

State the goals and design principles behind Assertify.

5.2. Implementation Details and Wrappers

TODO

Provide technical details on how Assertify wraps tests and improves output.

5.3. Error Message Improvements (Before and After Comparisons)

TODO

Compare previous error messages with Assertify's outputs.

5.4. Usability and Design Considerations

TODO

Discuss design decisions to enhance clarity and usability for students.

6. TODO: Further Methodology (unclear what comes next)

TODO

(1) VS Code Extension

(2) ExClaim Embedding

7. Evaluation and Results

TODO

Apparently there is something to do.

7.1. Evaluation Methodology

TODO

Explain how you assessed the effectiveness of Assertify.

7.2. Metrics for Success

TODO

Describe the criteria used to measure improvements.

7.3. Quantitative Analysis

TODO

Present the data and statistical results.

7.4. Qualitative Insights

TODO

Share insights from student feedback or observations.

7.5. Comparison to Previous Results

TODO

Highlight improvements compared to the baseline semester data.

8. Conclusion

TODO

Apparently there is something to do.

8.1. Summary of Contributions

TODO

Recap your key contributions.

8.2. Key Findings

TODO

Highlight the most important results and insights.

8.3. Suggestions for Future Work

TODO

Provide ideas for continued research and development.

9. Discussion

TODO

Apparently there is something to do.

9.1. Interpretation of Results

TODO

Analyze what the results reveal about error messages and learning.

9.2. Impact on Learning Process

TODO

Discuss how Assertify influenced students' debugging and comprehension.

9.3. Strengths and Limitations of Approach

TODO

Identify the strengths and potential weaknesses of the approach.

9.4. Recommendations for Future Development

TODO

Suggest possible extensions or refinements.

List of Figures

A. My Code

TODO

Put detailed code here.

B. My Ideas

TODO

Ideas??

Bibliography

[Tol12] J.R.R. Tolkien. *The Lord of the Rings: One Volume*. The Lord of the Rings. Houghton Mifflin Harcourt, 2012. ISBN: 9780547951942.